

BOOTCAMP FRONTEND & REACT · BLOQUE REACT · CLASE 3

La página reacciona

Remate de listas y condicionales — y las dos piezas del día: eventos y useState. El carrito nace hoy.

4 h 15 · L-M-V

Retomamos donde cortamos

STEERCL

SIN REPASOS LARGOS: RETOMAMOS Y SEGUIMOS

El plan de hoy

0. Reapertura

10'

el punto de corte + tareas destacadas

A. Remate: listas y condicionales

0-55'

solo lo pendiente · diapos: mazo de la clase 2 (láminas 7-10)

B. Eventos + useState

~95'

onClick · la memoria del componente · el carrito nace

→ Si el flujo se agota

—

seguimos con el material de la clase 4

C. Cierre + 5 tareas

25'

push demostrado, dudas y anticipo

**Receso de 15 min** alrededor de las 2:00 · verde = **el corazón del día**

DEL ADDEVENTLISTENER AL PROP

onClick

FUNDAMENTOS

querySelector para encontrar

addEventListener('click', ...)

delegación para no colgar cuarenta



REACT

el listener se declara **en el JSX**

como un prop, en camelCase: onClick

la delegación la hace React por dentro

```
<button onClick={manejar}>Agregar al carrito</button>
```

La función que le pasas se llama **manejador**: una función tuya, de las de siempre.

LA TRAMPA DE LOS PARÉNTESIS

La función, no su llamada

onClick={saludar}

le pasas LA FUNCIÓN — React la ejecuta cuando llega el clic

onClick={saludar()}

la ejecutas TÚ, durante el render — y le pasas el resultado: nada

```
// TypeScript ni compila la versión mala:  
Type 'void' is not assignable to type 'MouseEventHandler...'
```

¿Con argumento? Una flecha que la envuelve: `onClick={() => saludar()}` — la flecha ES la función entregada.

LA DEMO DEL DÍA

El clic que **no repinta**

```
let contador = 0;  
  
<button onClick={() => { contador = contador + 1; console.log(contador); }}>  
  Clics: {contador}  
</button>
```

consola: 1, 2, 3...

la variable **SÍ** cambia

pantalla: 0

nadie le pidió a React que repinte

Falta una caja que haga las dos cosas: **guardar el dato Y avisar**. La llevas leyendo desde la tarea de la clase 1.

LA MEMORIA DEL COMPONENTE

useState, desarmado

```
const [contador, setContador] = useState(0);
```

contador

el VALOR — de solo lectura, fresco en cada render

setContador

la FUNCIÓN para cambiarlo: guarda el dato Y pide el repintado

useState(0)

el valor INICIAL — y los corchetes son la desestructuración de array de JS moderno

PALABRA NUEVA PARA EL VOCABULARIO DEL BLOQUE

Esto es un **hook**

qué es

una función de React que se **engancha** a tu componente para darle un poder. Todas empiezan con use. Esta le da **memoria**; vendrán más.

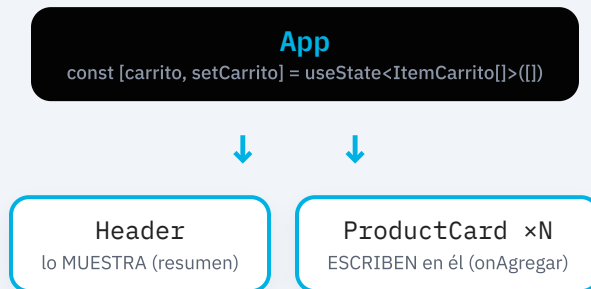
LA regla

se llaman **arriba del componente, siempre** — nunca dentro de un `if`, un `map` o una función anidada. React los identifica por orden de llamada.

Cada render es una foto. `setContador` no modifica la constante: pide a React **re-ejecutar** la función, y en esa nueva ejecución `useState` entrega el valor nuevo. Constantes frescas, derivados recalculados: `UI = f(datos)`, con el mecanismo a la vista.

DÓNDE VIVE EL ESTADO

¿De quién es el carrito?



Regla: el estado vive en el **ancestro común** de quienes lo usan. Y antes que la pantalla, el contrato y la lógica: `ItemCarrito` en `tipos.ts` y `carrito.ts` con funciones **puras**.

POR QUÉ LA INMUTABILIDAD ES REQUISITO

Referencia nueva = repintado

```
export function agregarItem(items: ItemCarrito[], producto: Producto): ItemCarrito[] {  
  const existente = items.find((i) => i.id === producto.id);  
  if (existente) {  
    return items.map((i) => (i.id === producto.id ? { ...i, cantidad: i.cantidad + 1 } :  
i));  
  }  
  return [...items, { id: producto.id, nombre: producto.nombre, precio: producto.precio,  
cantidad: 1 }];  
}
```

`items.push(...)` jamás: React decide si repinta comparando **por referencia**. Mismo array = quizá no se entera.

El estado no se muta — se reemplaza. En fundamentos era disciplina; aquí es requisito.

HASTA DONDE LLEGAMOS HOY

Lo de hoy

onClick

eventos como props en camelCase;
se pasa **la función**, nunca su llamada

useState

guarda el dato Y pide el repintado —
la variable normal no repinta

hook

función use... de React; arriba del
componente, jamás en un `if`

cada render, una foto

constantes frescas, derivados
recalculados — `UI = f(datos)`

ancestro común

el carrito vive en App y baja por
props — incluidas props-función

no se muta

`carrito.ts` devuelve arrays
nuevos: React compara por
referencia

CRITERIO DE ENTREGA: NPM RUN BUILD SIN ERRORES + PUSH

5 tareas

FÁCIL

Replica la clase, hasta donde llegamos: el carrito naciendo en tu tienda, con la grabación al lado. Es LA tarea.

INTERMEDIO

El agotado completo: etiqueta de últimas unidades, tarjeta apagada (`opacity-50`) y botón deshabilitado — los tres estados visuales del stock, coherentes.

INTERMEDIO

Quitar del carrito: tu `quitarItem(filter)` conectado a un botón `X` por fila. Si no llegamos a la lista en clase, escríbela tú: un `map` sobre `carrito` con su `key`.

DIFÍCIL

Cantidades que suben y bajan: escribe `cambiarCantidad(items, id, cantidad)` — pura, inmutable, y en cantidad 0 quita la fila — y conéctala a botones `+` y `-`.

DIFÍCIL

La forma funcional: investiga `setCarrito((prev) => ...)` en `es.react.dev` y comenta qué problema resuelve y cuándo la usarías.

Próxima clase — retomamos donde cortamos

Los filtros cobran vida

```
// se remata el carrito: quitar, vaciar, mostrar/ocultar...  
const [categoriaActiva, setCategoriaActiva] = useState('todas')  
const visibles = productos.filter(...) // derivado, no estado
```

La categoría con clics, el buscador que filtra mientras escribes — y la idea que ordena todo React: **guarda lo mínimo, deriva el resto**. Tu campo categoría de la tarea entra en acción.

Trae las tareas hechas y el build en cero. Nos vemos.